第四届eBPF开发者大会

The 4th eBPF Developer Conference (eBPFDC 2026)

陕西 · 西安　　2026年4月18日

https://www.ebpftravel.com

第四届 eBPF开发者大会

The 4th eBPF Developer Conference (eBPFDC 2026)

KernelScript:面向eBPF全场景开发的编程语言

KernelScript

陕西·西安 2026年4月18日

什么是 KernelScript?

KernelScript是一种为基于eBPF的内核定制化设计的一种编程语言。

它打破了传统壁垒，允许开发者在**单一代码库**中无缝编写eBPF程序、用户空间控制逻辑以及内核空间功能。

通过内置对内核特性的深度融合，KernelScript将繁杂的系统级编程转化为高效、安全的现代开发语言。

代码仓库：https://github.com/multikernel/kernelscript

核心优势：统一与安全

单文件多目标编译

开发者只需添加简单的属性标签（如@xdp, @kfunc），编译器即可自动识别并将其分别编译为eBPF、用户空间和内核模块的目标代码。彻底消除了维护多个内核项目的不便。

程序生命周期安全

KernelScript拥有专门为eBPF验证器优化的强类型系统。它能够在编译期捕捉不兼容的类型，例如kprobe函数原型检查，从而在代码编译时就发现和目标内核版本不一致的类型。

关键特性：自动尾调用编排

核心理念：把原本eBPF手工化的尾调用编排过程转化成语言的一部分。

语言级能力抽象：开发者只需要写普通的尾调用，编译器会根据上下文把它识别成安全的尾调用，并自动生成底层所需的map、索引和初始化逻辑。

彻底消除样板代码：编译器自动完成PROG_ARRAY定义、索引分配与bpf_tail_call()注入。自动生成用户态侧的加载与初始化逻辑，并内置错误处理。

严谨的编译期校验：严格限制调用条件，只有同属带程序属性（如@xdp）的函数之间才能跳转。强制校验程序类型的兼容性、上下文参数及返回类型，从源头杜绝不确定行为。

典型适用场景：突破单一程序指令限制，非常适合“分发”和“阶段切换”。完美适配XDP包处理流水线（分类、协议解析、策略执行拆分）及复杂match分支分发。

关键特性：基于 BTF 的类型安全

核心理念：从BTF中自动提取内核结构体以及函数原型。

语言级能力抽象：对应的就是 @probe("func_name")、@tracepoint("category/event") 。开发者写的是“我要探测哪个函数”或者“我要挂在哪个 tracepoint 上”，而编译器负责把它与同具体的内核函数匹配起来。

彻底消除样板代码：不再让开发者手工维护大量与内核结构体、函数签名相关的声明，编译器自动从 BTF 中提取需要的内核结构体和函数原型。

严谨的编译期校验：不只是减少样板代码，而是把很多类型错误提前到编译阶段发现。在 BPF verifier 看到目标代码之前，编译器已经以先完成一轮类型一致性校验。

典型适用场景：tracepoint、kprobe等深度绑定内核函数的场景，struct_ops复杂数据结构，在编译时就可以捕获类型错误。

关键特性：透明动态指针

核心理念：把容易出错但模式化的动态指针细节隐藏在编译阶段完成。

屏蔽底层复杂细节：开发者无需显式声明动态指针类型，无需手动计算偏移，无需关心reserve/submit函数。直接使用普通指针解引用、结构体字段读写和常规的Ring Buffer操作。

智能匹配安全接口：编译器会根据数据的来源（如数据包、Map或Ring Buffer），自动替换为最合适的动态指针函数（如bpf_dynptr_from_xdp）。

兼顾代码易读与安全：既保证了业务代码的清晰简洁，又让最终生成的程序符合 eBPF 验证器的严格校验，从源头杜绝内存越界风险。

典型适用场景：非常适合处理Ring Buffer事件对象、对Map数据进行结构化读写，以及在XDP/TC网络程序中灵活解析不定长的数据包。

关键特性：内置全局 map 变量

核心理念：让跨程序的map共享不再只是ebpf技巧，而成为语言的一部分。

状态共享语义抽象：替代传统开发中的C宏、.maps section和文件描述符传递。只需在代码顶层声明，所有的eBPF程序（如XDP、TC）即可直接读写该Map。

编译期全链路自动化：自动完成内核空间与用户空间的对接。编译器会自动生成Map的创建、文件描述符获取，以及多程序协同加载的完整代码。

极简的持久化管理：引入pin 、var关键字，编译器即可自动生成bpffs文件系统路径（/sys/fs/bpf/...）的绑定逻辑，实现数据的跨进程与跨程序生命周期。

典型适用场景：适用于多个eBPF程序共享统计计数器、维护跨程序的状态机，以及使用Ring Buffer构建统一的事件分发通道。

关键特性：内置 kfunc 支持

核心理念：把eBPF、kfunc、用户态程序统一成一个完整的程序。

自定义内核能力（@kfunc/@private）：开发者可直接在代码中编写kfunc。编译器会自动将其打包为标准内核模块并处理BTF注册，同时支持通过@private隐藏模块内部代码。

复用现有内核能力（extern）：无需编写内核模块，只需声明目标函数，即可在eBPF程序中直接调用当前内核已存在的原生kfunc。

编译期全链路自动化：编译器会自动分析eBPF程序与kfunc之间的依赖关系，并在用户空间代码中自动生成正确顺序的模块加载与卸载逻辑。

典型适用场景：适用于突破eBPF Verifier限制执行复杂高权限逻辑，或在单一工程项目中统一管理用户态、eBPF态及内核态的全部代码。

全自动化的软件工程构建

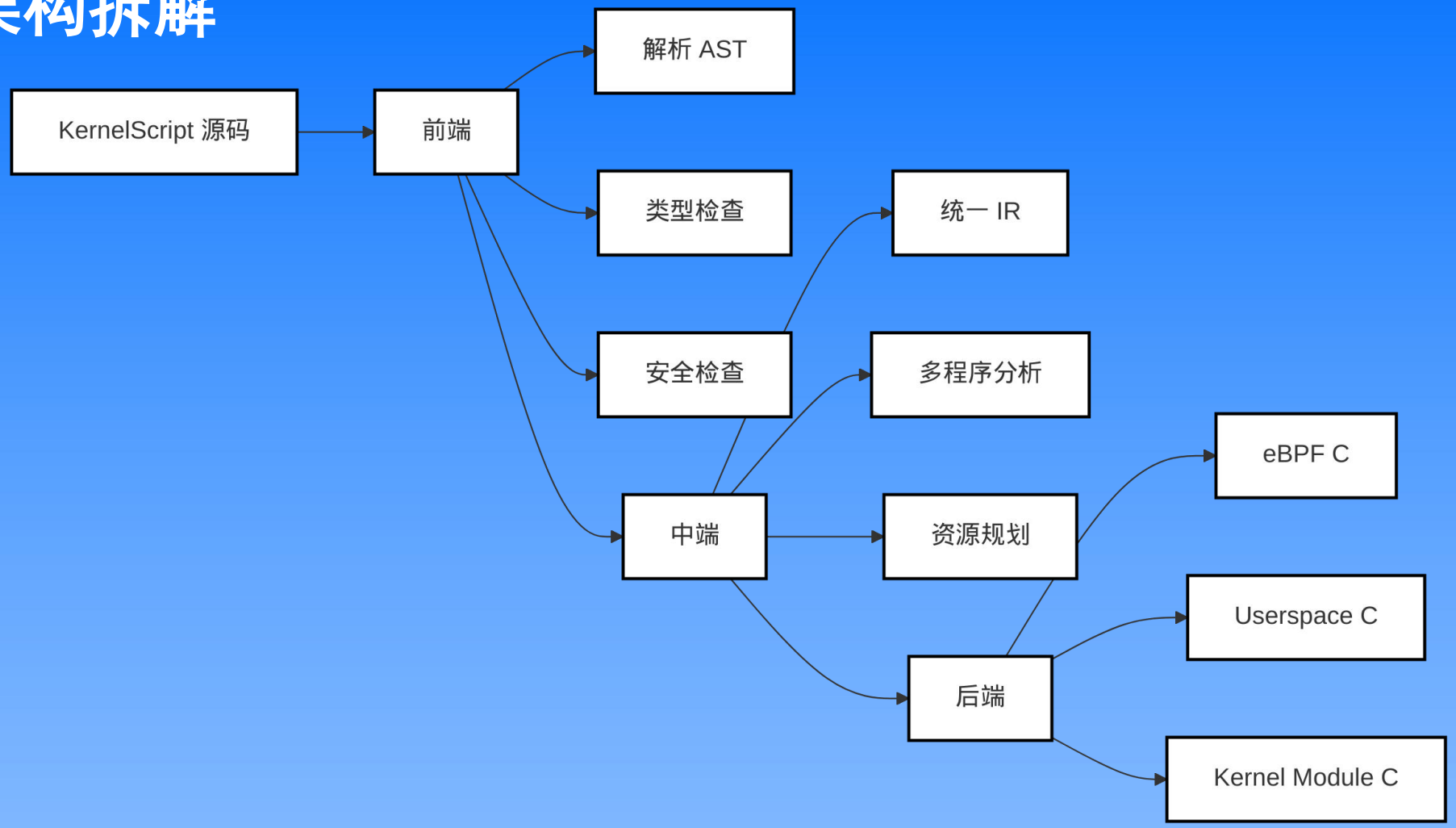
告别手动配置： 不再需要手动维护复杂的Makefile和构建脚本。

一键生成项目： 从单一的KernelScript源文件自动生成完整的项目架构。

智能加载器： 自动生成对应目标环境的用户空间加载器。

兼容特定内核版本： 类型系统基于BTF，确保生成的字节码兼容严苛的eBPF验证器。

编译器架构拆解


KernelScript 源码
前端
解析 AST
类型检查
安全检查
中端
统一 IR
多程序分析
资源规划
后端
eBPF C
Userspace C
Kernel Module C

编译器架构总结

主要目标	把源码变成带语义约束的系统描述	把语义模型降级成统一 IR 并做系统级分析	把统一 IR 生成为多个可编译目标
输入	.ks 源码、import、include、builtin 声明	typed AST、多程序分析结果、符号信息	IR、tail call 分析、ring buffer 分析、上下文信息
核心工作	解析、依赖补全、符号表、类型检查、安全检查	lowering、控制流组织、资源分析、跨程序协调	eBPF C 生成、userspace C 生成、kernel module C 生成
关注重点	语义正确、执行域合法、提前报错	多程序建模、资源规划、IR 组织	输出物正确、符合 verifier / libbpf / kfunc 约束
典型问题	函数能否在当前域调用、类型是否匹配、map 操作是否合法	tail call 如何分配、共享资源如何建模、程序如何协同	section 如何生成、attach 逻辑如何生成、是否需要额外模块
代表模块	lexer.mll、parser.mly、ast.ml、type_checker.ml、safety_checker.ml	ir.ml、ir_generator.ml、multi_program_analyzer.ml、tail_call_analyzer.ml	ebpf_c_codegen.ml、userspace_codegen.ml、kernel_module_codegen.ml
输出结果	带类型和语义约束的程序表示	可被多个后端消费的统一 IR	.ebpf.c、.c、.mod.c、可选测试与构建文件

适用场景

高性能网络（XDP）

利用KernelScript快速构建防御DDoS攻击、负载均衡及自定义数据包路由的高性能网络过滤器。

系统可观测性

通过类型安全的探针（Kprobes/Tracepoints）深入洞察系统性能瓶颈，无需担心内核崩溃。

内核安全扩展

借助对LSM和kfuncs的无缝支持，实现极低开销的自定义内核级安全策略与访问控制。

XDP包过滤案例

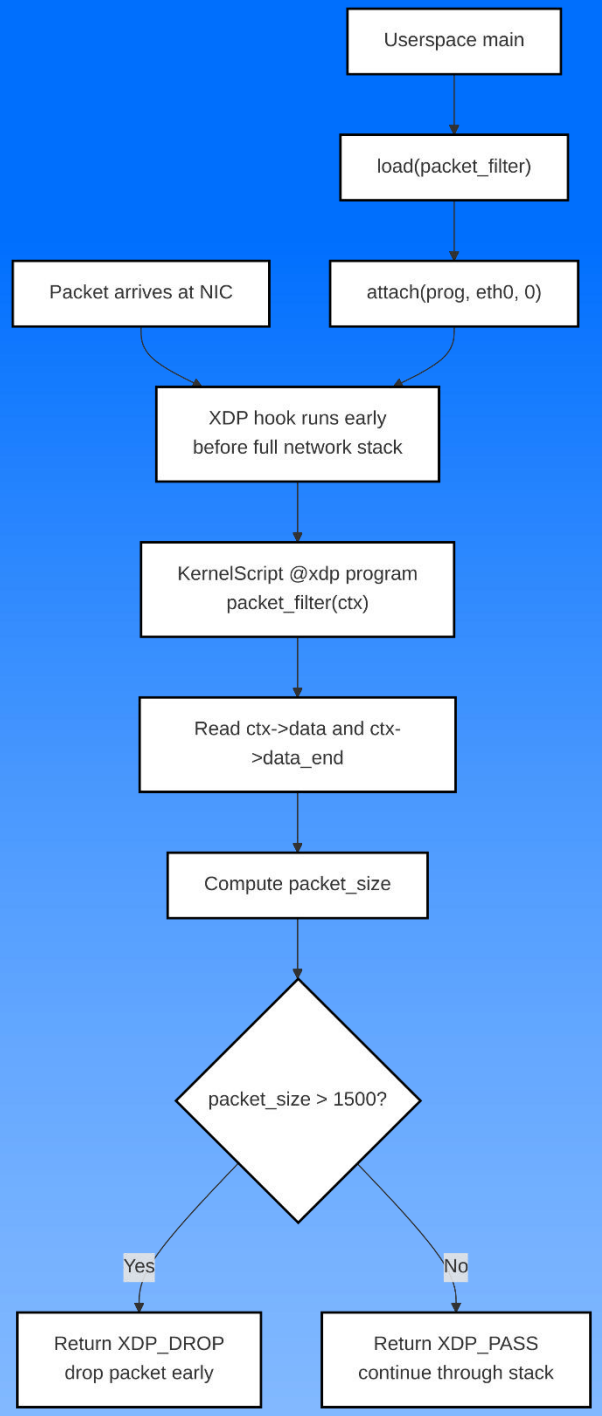
XDP 包过滤场景

- 场景目标：在网卡收包最早阶段快速判断是否放行
- 典型用途：基础DDoS防护、报文长度过滤、协议预筛选
- 关键要求：越早丢包，越少进入内核协议栈，系统开销越低
- 对编译器的要求：生成验证器友好、attach简单、用户态加载流程清晰

KernelScript让XDP程序写法接近“带类型约束的策略描述”；既保留eBPF的执行语义，又减少手写C和样板代码负担；适合把简单过滤、早期拦截、包路径控制快速落地

XDP包过滤案例

内存级直通带来的零拷贝防御



陕西 · 西安　2026年4月18日

XDP包过滤案例

XDP 例子在 KernelScript 里的表达

- 使用@xdp声明程序入口，语义直接对应XDP执行域
- 参数显式写成 ctx: *xdp_md，和内核上下文类型一致
- 返回值使用xdp_action，直接表达XDP_PASS / XDP_DROP
- 用户态主函数中通过 load、attach、detach 完成生命周期管理

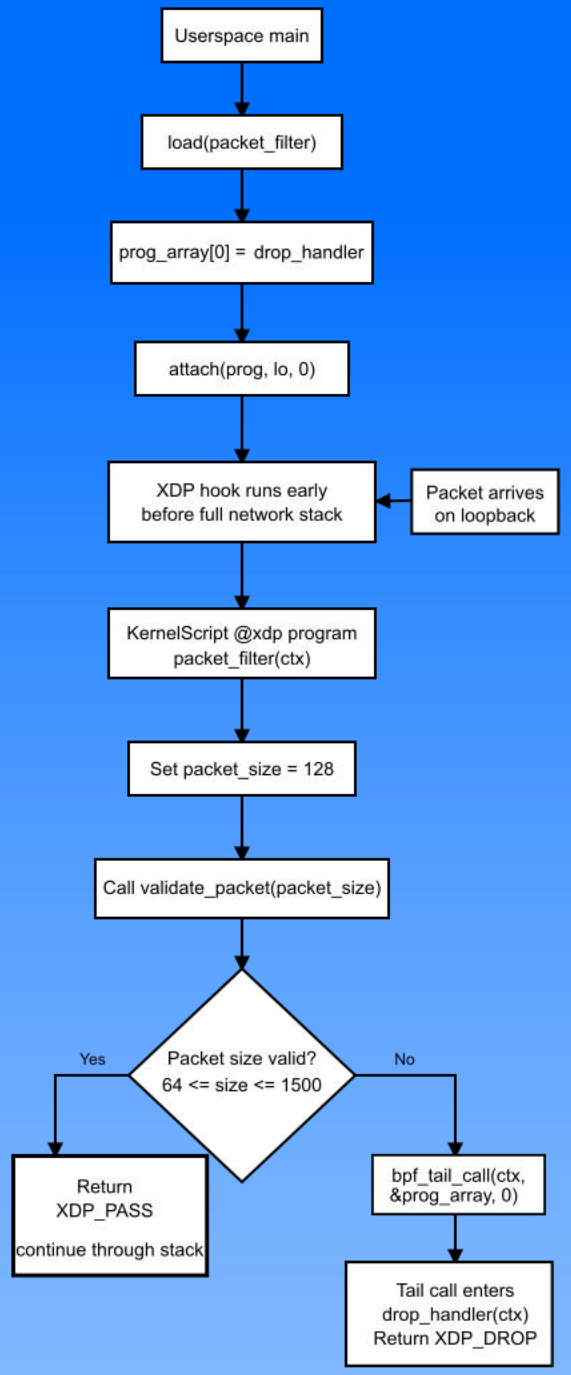
代码要点

- 先读取data和data_end，计算报文大小
- 超过阈值就丢弃，否则放行
- 业务逻辑很短，但已经覆盖了“内核态处理 + 用户态加载”完整链路

Demo Code: kernelscript/examples/packet_filter.ks

XDP包过滤案例

基于尾调用（Tail Call）的模块化设计



XDP包过滤案例

XDP 例子在 KernelScript 里的表达

- 声明一个类型为BPF_MAP_TYPE_PROG_ARRAY的 Map（如prog_array），这是实现尾调用的核心数据结构，用于存放目标程序的句柄
- 定义多个独立的 XDP 程序，两者具有相同的上下文参数ctx: *xdp_md
- 在主程序中，通过bpf_tail_call(ctx, &prog_array, 0)将执行权无缝移交给数组索引 0 处对应的子模块

代码要点

- packet_filter仅负责基础校验，一旦发现异常，不直接丢弃，而是触发尾调用
- 尾调用一旦成功触发，当前程序的执行立即终止，避免了传统函数调用的栈开销。KernelScript 让这种底层的控制流转移变得像调用普通函数一样简单

Demo Code: kernelscript/examples/tail_call.ks

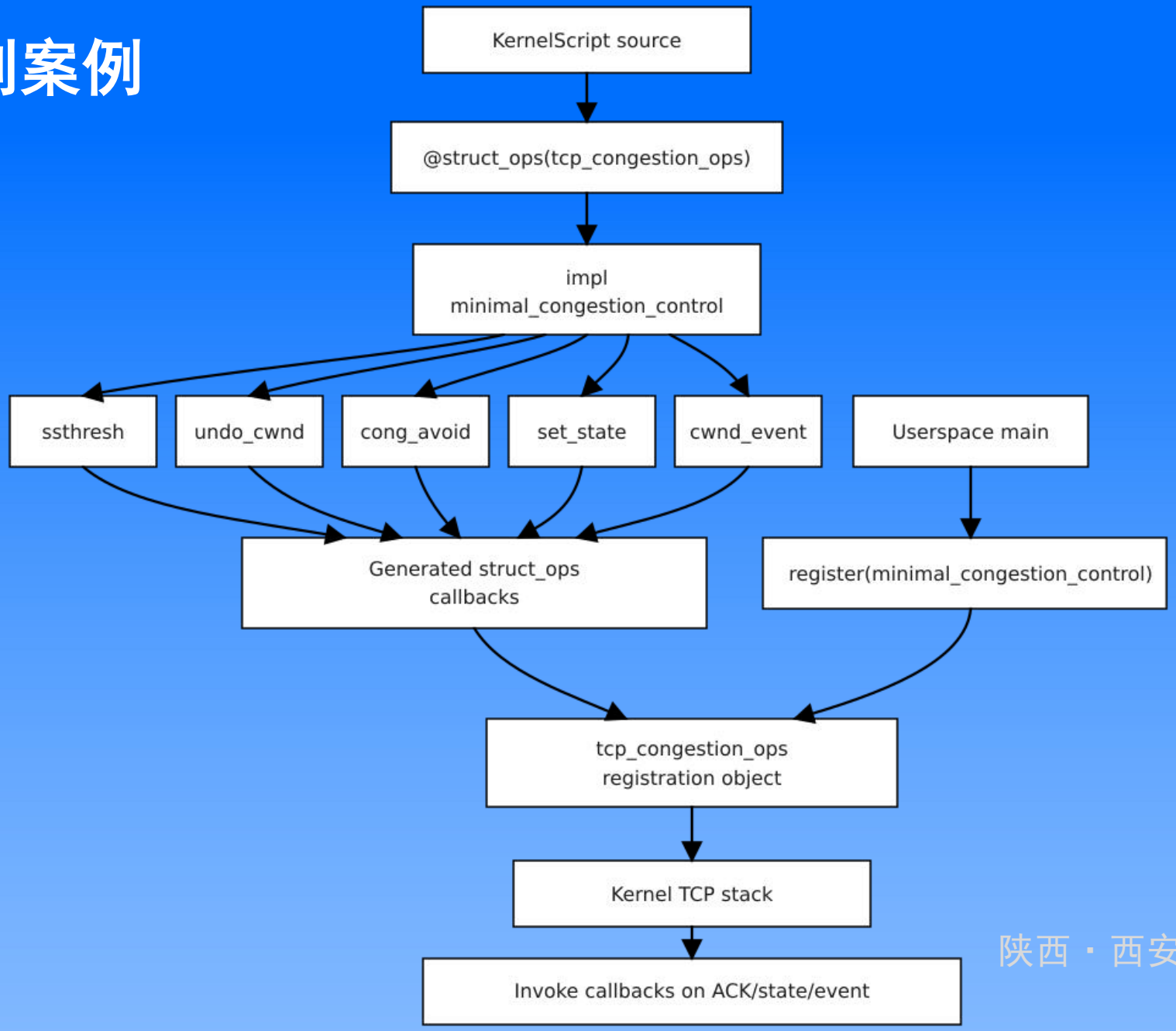
TCP拥塞控制案例

TCP 拥塞控制场景

- 场景目标：把一组回调函数注册为TCP拥塞控制算法
- 典型用途：自定义拥塞窗口增长、状态切换处理、事件回调
- 和普通XDP程序不同，它不是“处理一个包”，而是“定义一类协议行为”
- 对编译器的要求：不仅要生成函数，还要生成可注册的struct_ops映射关系

KernelScript 把struct_ops从“繁琐结构体装配”提升为“语义化接口实现”；对协议扩展类场景更友好，代码结构也更接近开发者思维模型；适合展示编译器不只会生成普通eBPF程序，也能覆盖内核可插拔机制

TCP拥塞控制案例


KernelScript source
@struct_ops(tcp_congestion_ops)
impl
minimal_congestion_control
ssthresh
undo_cwnd
cong_avoid
set_state
cwnd_event
Userspace main
Generated struct_ops
callbacks
register(minimal_congestion_control)
tcp_congestion_ops
registration object
Kernel TCP stack
Invoke callbacks on ACK/state/event

TCP拥塞控制案例

TCP 拥塞控制在KernelScript里的表达

- 通过 @struct_ops("tcp_congestion_ops") 指定目标内核接口
- 使用impl minimal_congestion_control组织回调函数
- 这些函数直接写在同一个块里
- 用户态主函数只需要register()即可完成注册

代码要点

- impl块的写法比手动构造结构体更直观
- 已实现的函数会成为struct_ops对应回调

Demo Code:
kernelscript/examples/struct_ops_simple.ks

KernelScript：重塑 eBPF 开发

统一开发范式：单一代码库无缝融合 eBPF、用户态与内核态逻辑。
极致语言抽象：编译器自动接管尾调用、动态指针与全局 Map，彻底消除底层样板代码。
编译期安全检查：依托强类型系统与自动 BTF 提取，将状态异常与越界风险拦截在运行前。
全场景业务赋能：完美适配高性能网络（XDP）、系统可观测性探针与极低开销的安全扩展（LSM）。

代码仓库：https://github.com/multikernel/kernelscript

KernelScript 打破了 eBPF、用户态和内核态之间的技术壁垒，实现了“单一代码库，全栈驱动开发”。

谢谢大家